KIYARI SMART VOICE COMPANION
Technical Project Architecture & Future Portability Guide

1. Project Overview & Architectural Design

The KIYARI Smart Voice Companion is a customized, completely offline hardware-software voice system designed to serve as the referee, interactive helper, and rule database for the agriculture board game 'KIYARI'. The system consists of an ESP32-S3 hardware satellite connected via real-time WebSockets to a local PC backend acting as the AI processing brain. This architecture enables low-latency hold-to-talk speech recognition, local large language model reasoning, and dynamic bilingual voice feedback.

2. Custom Firmware Framework & Drivers

The firmware is developed using ESP-IDF v5.5.4 (Espressif IoT Development Framework) in C, leveraging FreeRTOS for scheduling real-time tasks. Below are the hardware controllers and drivers implemented:

* **I2S (Inter-IC Sound) Protocol:** Used for both audio input and output. The INMP441 digital microphone uses an I2S RX channel configured for 16kHz sample rate, 16-bit mono PCM. The MAX98357A DAC/Amplifier uses an I2S TX channel to decode PCM16 bytes from the server and drive the analog speaker.

* **I2C (Inter-Integrated Circuit):** Drives the SSD1306 OLED display (128x64 pixels) to show state updates such as 'Connected', 'Ready', 'Listening...', 'Thinking...', and 'Speaking...'.

* **GPIO Control & Interrupts:** GPIO 0 is configured to poll the physical BOOT button state, while GPIO 48 drives the Status RGB LED.

* **Real-Time Streaming Loop:** When the BOOT button is held down, the microphone is activated, and a FreeRTOS task packages the raw audio into chunks and streams them instantly over WebSocket. When released, streaming stops and a trigger is sent.

* **ESP WebSocket Client:** Establishes a persistent TCP WebSocket connection to the backend server. It handles control messages (JSON) and audio streams (binary) simultaneously.

3. Offline AI Backend Pipeline

The backend is built in Python 3.13 using FastAPI. It functions entirely without internet connectivity, processing speech and text locally:

* **FastAPI WebSockets:** Accepts the persistent connection from the ESP32. It routes incoming voice data to the STT engine, passes transcripts to the LLM, and streams synthesized speech back.

* **Faster-Whisper (STT):** Converts the raw PCM16 audio received from the ESP32 into text. It runs locally on the PC GPU/CPU and supports multilingual speech (automatic English/Hindi detection).

* **Qwen 2.5 7B Instruct (LLM):** Loaded locally via LM Studio. The model is injected with a system prompt containing the compiled rules of KIYARI, allowing it to act as an expert referee.

* **Piper (TTS):** A fast, local neural text-to-speech synthesizer. It dynamically switches between a native Hindi voice (hi_IN-pratham-medium) and an English voice (en_US-lessac-medium) based on output text.

* **Digital Audio Processor:** Intercepts user prompts containing volume commands. It scales the 16-bit PCM amplitude values digitally (multiplier 0.1 to 1.2) and applies clipping protection (np.clip) to keep audio output distortion-free.

4. Future Scope: Complete Portability (No Laptop or Wi-Fi)

To make the KIYARI Voice Assistant fully portable and standalone - without requiring a laptop, router, or external Wi-Fi - the following hardware and software upgrades can be implemented in future iterations:

* **On-Chip Wake Word & Command Recognition:** Replace the server-side processing with on-chip wake-word models. By using the ESP-Skainet framework (Espressif's voice assistant development framework) or TensorFlow Lite Micro, the ESP32-S3 can run light voice commands locally.

* **Edge LLM & Speech Accelerator:** Integrate a hardware neural accelerator (like the K210, Kendryte, or a small Raspberry Pi CM4) into the enclosure. This allows running highly compressed 1B parameter LLMs (like TinyLlama) and lightweight local TTS on-the-edge, fitting inside a small handheld toy.

* **Wi-Fi-Free Hotspot Mode (Access Point):** Configure the ESP32-S3 to start in Wi-Fi Access Point (AP) mode. It creates its own local Wi-Fi hotspot. A smartphone can connect to this network and host the backend server, making the laptop redundant while outdoors.

* **Offline MicroSD Storage:** Add an SPI MicroSD card slot to the ESP32. The game manuals, lookup tables, and pre-recorded audio responses can be stored on the SD card. Instead of generating speech dynamically, the ESP32 can play back pre-recorded voice clips based on simple index matches.

* **Battery & Portable Power Integration:** Integrate a LiPo battery (e.g., 3.7V 1000mAh) with a TP4056 charging board and a boost converter (to step up to 5V). This allows the entire KIYARI companion to operate completely unplugged in a custom 3D-printed enclosure.

Summary of Frameworks & Technologies Used:

Hardware: ESP32-S3 DevKitC-1
Firmware Framework: ESP-IDF v5.5.4 (C Language, FreeRTOS)
Backend Web Framework: FastAPI (Python 3.13)
Speech-to-Text: Faster-Whisper (Local ONNX)
Language Model: Qwen 2.5 7B Instruct (LM Studio)
Text-to-Speech: Piper TTS (Local ONNX)
Audio Drivers: Standard I2S drivers (Microphone & Speaker DAC)